

MAT41-3

NN & Deep Learning Math

Forward propagation, backpropagation, softmax, and training dynamics

The mathematical structure of feedforward neural networks: how they compute, how they learn, and why training succeeds or fails.

Albert School — 30 hours

Contents

1	Setup	2
2	Forward propagation	2
3	Why activations matter	3
4	Backpropagation	3
5	Softmax	4
6	Cross-entropy loss	5
7	Residual connections	5
8	Layer normalization	6
9	Vanishing and exploding gradients	6
10	Regularization	7

1 Setup

A feedforward neural network with L layers maps an input vector to an output vector by alternating affine maps and component-wise nonlinearities. We use vectors and matrices, matrix products and transposes (MAT21-1), the scalar product and symmetric/positive-definite matrices (MAT31-1), the chain rule and convexity (MAT11-1), and probability distributions, expectation, and maximum likelihood (MAT31-2).

Throughout, $a^{(0)} = x$ is the input, $W^{(l)}$ and $b^{(l)}$ are the weight matrix and bias vector of layer l , and σ is an activation function applied component-wise. The output of the last layer is the network's prediction.

2 Forward propagation

Definition 1 (Affine map of a layer) Layer l computes the **pre-activation**

$$z^{(l)} = W^{(l)}a^{(l-1)} + b^{(l)},$$

where $W^{(l)} \in \mathbb{R}^{n_l \times n_{l-1}}$, $a^{(l-1)} \in \mathbb{R}^{n_{l-1}}$, and $b^{(l)} \in \mathbb{R}^{n_l}$.

Definition 2 (Activation) The **activation** (post-activation) of layer l is obtained by applying a function σ to each component of $z^{(l)}$:

$$a^{(l)} = \sigma(z^{(l)}), \quad (a^{(l)})_i = \sigma((z^{(l)})_i).$$

Definition 3 (Forward pass) Given an input x , the **forward pass** is the composition

$$a^{(0)} = x, \quad z^{(l)} = W^{(l)}a^{(l-1)} + b^{(l)}, \quad a^{(l)} = \sigma(z^{(l)}), \quad l = 1, \dots, L,$$

and the output is $\hat{y} = a^{(L)}$.

To execute a forward pass by hand: for each layer in order, form the matrix product $W^{(l)}a^{(l-1)}$, add the bias $b^{(l)}$ to get $z^{(l)}$, then apply the activation component-wise to get $a^{(l)}$; the activation of the final layer is the output.

3 Why activations matter

Proposition 4 (Linear collapse) If every activation is the identity, $\sigma = \text{id}$, then the network computes a single affine map

$$a^{(L)} = W a^{(0)} + b$$

for some matrix W and vector b , namely

$$W = W^{(L)} W^{(L-1)} \dots W^{(1)}.$$

Proof. With $\sigma = \text{id}$ we have $a^{(l)} = W^{(l)} a^{(l-1)} + b^{(l)}$ at every layer. Substituting layer by layer,

$$\begin{aligned} a^{(2)} &= W^{(2)} (W^{(1)} a^{(0)} + b^{(1)}) + b^{(2)} \\ &= W^{(2)} W^{(1)} a^{(0)} + (W^{(2)} b^{(1)} + b^{(2)}). \end{aligned}$$

Iterating to layer L gives $a^{(L)} = W a^{(0)} + b$ with $W = W^{(L)} \dots W^{(1)}$ and b a corresponding combination of the biases. $\square$

A product of matrices is a single matrix, so without nonlinear activations an arbitrarily deep network represents only affine functions — exactly the class a single layer already represents. Depth adds nothing in this case; the nonlinear activations are what make depth expressive.

4 Backpropagation

Training minimizes a scalar loss L computed from the output $a^{(L)}$ and the target y . Gradients of L with respect to all weights and biases are computed by the chain rule applied to the computational graph of the forward pass.

Definition 5 (Error vector) The **error** of layer l is the gradient of the loss with respect to that layer's pre-activation,

$$\delta^{(l)} = \frac{\partial L}{\partial z^{(l)}} \in \mathbb{R}^{n_l}.$$

Proposition 6 (Output-layer error) At the last layer,

$$\delta^{(L)} = \frac{\partial L}{\partial a^{(L)}} \odot \sigma'(z^{(L)}),$$

where $\odot$ is the component-wise (Hadamard) product and σ' is applied component-wise.

Proposition 7 (Backpropagation recurrence) For $l = L - 1, \dots, 1$, the error propagates backward through the transposed weight matrix:

$$\delta^{(l)} = \left((W^{(l+1)})^T \delta^{(l+1)} \right) \odot \sigma'(z^{(l)}).$$

Proof. By the chain rule, $\delta^{(l)} = (\partial z^{(l+1)} / \partial z^{(l)})^T \delta^{(l+1)}$. Since $z^{(l+1)} = W^{(l+1)} \sigma(z^{(l)}) + b^{(l+1)}$, the Jacobian $\partial z^{(l+1)} / \partial z^{(l)} = W^{(l+1)} \text{diag}(\sigma'(z^{(l)}))$. Transposing and multiplying by $\delta^{(l+1)}$ gives the stated formula, the diagonal matrix acting as the component-wise product with $\sigma'(z^{(l)})$. $\square$

Proposition 8 (Weight and bias gradients as outer products) Once $\delta^{(l)}$ is known,

$$\frac{\partial L}{\partial W^{(l)}} = \delta^{(l)} (a^{(l-1)})^T, \quad \frac{\partial L}{\partial b^{(l)}} = \delta^{(l)}.$$

The weight gradient is the **outer product** of the layer's error with the incoming activation.

Backpropagation procedure.

1. Run a forward pass, storing every $z^{(l)}$ and $a^{(l)}$.
2. Compute $\delta^{(L)} = (\partial L / \partial a^{(L)}) \odot \sigma'(z^{(L)})$.
3. For $l = L - 1, \dots, 1$, set $\delta^{(l)} = ((W^{(l+1)})^T \delta^{(l+1)}) \odot \sigma'(z^{(l)})$.
4. Read off $\partial L / \partial W^{(l)} = \delta^{(l)} (a^{(l-1)})^T$ and $\partial L / \partial b^{(l)} = \delta^{(l)}$.

5 Softmax

Definition 9 (Softmax) The **softmax** function maps a vector of **logits** $z \in \mathbb{R}^K$ to a probability distribution $\hat{y} \in \mathbb{R}^K$ by

$$\hat{y}_i = \text{softmax}(z)_i = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}, \quad i = 1, \dots, K.$$

Each $\hat{y}_i > 0$ and $\sum_i \hat{y}_i = 1$.

Definition 10 (Temperature) With **temperature** parameter $T > 0$, the softmax is applied to z/T :

$$\hat{y}_i = \frac{e^{z_i/T}}{\sum_j e^{z_j/T}}.$$

As $T \rightarrow 0$ the distribution concentrates on the largest logit (sharp); as $T \rightarrow \infty$ it tends to the uniform distribution (flat).

Proposition 11 (Softmax Jacobian) The partial derivatives of the softmax outputs with respect to the logits are

$$\frac{\partial \hat{y}_i}{\partial z_j} = \hat{y}_i (\delta_{ij} - \hat{y}_j),$$

where δ_{ij} is 1 if $i = j$ and 0 otherwise. In matrix form,

$$J = \text{diag}(\hat{y}) - \hat{y} \hat{y}^T.$$

Proof. For $i = j$, differentiating $\hat{y}_i = e^{z_i}/S$ with $S = \sum_k e^{z_k}$ gives $\partial \hat{y}_i / \partial z_i = (e^{z_i} S - e^{z_i} e^{z_i}) / S^2 = \hat{y}_i (1 - \hat{y}_i)$. For $i \neq j$, $\partial \hat{y}_i / \partial z_j = -e^{z_i} e^{z_j} / S^2 = -\hat{y}_i \hat{y}_j$. Both cases are summarized by $\hat{y}_i (\delta_{ij} - \hat{y}_j)$. $\square$

6 Cross-entropy loss

Definition 12 (Cross-entropy) For a true distribution y and a predicted distribution $\hat{y}$ over K classes, the **cross-entropy loss** is

$$L = - \sum_{i=1}^K y_i \log \hat{y}_i.$$

When the true label is class c (so y is the one-hot vector with $y_c = 1$), this reduces to $L = -\log \hat{y}_c$.

The loss measures how wrong the predicted distribution $\hat{y}$ is relative to the true label y : it is small when $\hat{y}$ places high probability on the correct class and grows without bound as $\hat{y}_c \rightarrow 0$.

Proposition 13 (Connection to maximum likelihood) For a single example with true class c , the model assigns probability $\hat{y}_c$ to the observed label, so the log-likelihood is $\log \hat{y}_c = -L$. Over a dataset of independent examples,

$$\sum_{\text{examples}} \log \hat{y}_c = - \sum_{\text{examples}} L.$$

Minimizing total cross-entropy is therefore identical to maximizing the likelihood of the observed labels.

Proposition 14 (Gradient of softmax + cross-entropy) Let $\hat{y} = \text{softmax}(z)$ and $L = -\sum_i y_i \log \hat{y}_i$ with y a probability vector. Then

$$\frac{\partial L}{\partial z} = \hat{y} - y.$$

Proof. By the chain rule, $\partial L / \partial z_j = \sum_i (\partial L / \partial \hat{y}_i) (\partial \hat{y}_i / \partial z_j)$. Here $\partial L / \partial \hat{y}_i = -y_i / \hat{y}_i$, and using the softmax Jacobian $\partial \hat{y}_i / \partial z_j = \hat{y}_i (\delta_{ij} - \hat{y}_j)$,

$$\frac{\partial L}{\partial z_j} = - \sum_i \frac{y_i}{\hat{y}_i} \hat{y}_i (\delta_{ij} - \hat{y}_j) = - \sum_i y_i (\delta_{ij} - \hat{y}_j) = -y_j + \hat{y}_j \sum_i y_i.$$

Since $\sum_i y_i = 1$, this equals $\hat{y}_j - y_j$. □

The cancellation of $\hat{y}_i$ between the loss derivative and the Jacobian is why the softmax-with-cross-entropy gradient takes the simple form $\hat{y} - y$: predicted minus true. This is the $\delta^{(L)}$ used to start backpropagation when the network ends in a softmax with cross-entropy loss.

7 Residual connections

Definition 15 (Residual connection) A **residual** (skip) connection replaces a block f by

$$a = x + f(x),$$

adding the block's input back to its output. The block computes the **perturbation** $f(x)$ around the identity.

Proposition 16 (Gradient flow through a residual block) For $a = x + f(x)$, the Jacobian with respect to the input is

$$\frac{\partial a}{\partial x} = I + \frac{\partial f}{\partial x}.$$

Because the Jacobian of a residual block is the identity plus the block's own Jacobian, the backpropagated gradient contains an additive term that passes through unchanged. Stacking such blocks multiplies factors of the form $I + \partial f / \partial x$ rather than $\partial f / \partial x$ alone, so the gradient does not decay to zero merely by passing through many layers.

8 Layer normalization

Definition 17 (Layer normalization) Given a pre-activation vector $z \in \mathbb{R}^n$, **layer normalization** standardizes it across its components:

$$\mu = \frac{1}{n} \sum_{i=1}^n z_i, \quad s^2 = \frac{1}{n} \sum_{i=1}^n (z_i - \mu)^2, \quad \hat{z}_i = \frac{z_i - \mu}{\sqrt{s^2 + \varepsilon}},$$

then applies a learned scale γ_i and shift β_i :

$$\text{LN}(z)_i = \gamma_i \hat{z}_i + \beta_i.$$

The constant $\varepsilon > 0$ avoids division by zero.

Layer normalization forces every layer's input to have mean 0 and variance 1 (before the learned γ, β), keeping activations on a controlled scale throughout the network and so stabilizing training.

Remark Layer normalization computes μ and s^2 across the **features of a single example**, so it does not depend on other examples in a batch. **Batch normalization** instead computes these statistics across the **batch dimension**, for each feature separately; it therefore depends on batch composition and on having a batch at all, whereas layer normalization does not.

9 Vanishing and exploding gradients

The backpropagation recurrence multiplies by a factor involving $W^{(l+1)}$ and $\sigma'(z^{(l)})$ at each layer. Across L layers the gradient at an early layer is a product of L such factors.

Proposition 18 (Compounding across layers) If each backward factor scales the gradient by roughly a constant κ (governed by the size of the weights and of σ'), then after L layers the early-layer gradient scales like κ^L . Hence:

- if $\kappa < 1$ (small weights), $\kappa^L \rightarrow 0$ — **vanishing gradients**;
- if $\kappa > 1$ (large weights), $\kappa^L \rightarrow \infty$ — **exploding gradients**.

Vanishing gradients prevent early layers from learning (their updates are negligible); exploding gradients make training diverge (updates are enormous). Both worsen with depth, limiting how deep a network can be trained naively.

Mitigations.

- **Residual connections:** the additive identity term keeps a factor of 1 in the product, so the gradient does not decay merely by depth.
- **Careful initialization:** choosing the initial weight scale so that each backward factor is close to 1 (i.e. $\kappa \approx 1$) keeps κ^L from blowing up or collapsing across the layers.

10 Regularization

Overfitting occurs when a network fits the training data, including its noise, so closely that it generalizes poorly. Regularization controls model complexity to reduce overfitting.

Definition 19 (Weight decay (L2 regularization)) **Weight decay** adds a penalty proportional to the squared norm of the weights to the loss:

$$L_{\text{reg}} = L + \frac{\lambda}{2} \sum_l \|W^{(l)}\|^2,$$

with $\lambda > 0$. Its gradient contribution is $\lambda W^{(l)}$, so each update shrinks the weights toward 0:

$$W^{(l)} \leftarrow W^{(l)} - \eta \frac{\partial L}{\partial W^{(l)}} - \eta \lambda W^{(l)}.$$

By penalizing large weights, weight decay favours smaller-magnitude parameters, which produce smoother functions that are less able to fit noise.

Definition 20 (Dropout) **Dropout** randomly sets each activation to 0 with probability p during training, independently per unit and per example; the surviving activations are rescaled so the expected value is unchanged. At test time no units are dropped.

Dropout prevents units from co-adapting to specific other units, since any unit may be absent, forcing the network to learn redundant, more robust features.

Remark (Bias-variance tradeoff) An unregularized network has low bias but high variance (it fits noise); regularization increases bias slightly while reducing variance. Weight decay and dropout are controls on this tradeoff: increasing λ or p moves the model toward higher bias and lower variance, and the value minimizing generalization error balances the two.